DSD

Lab Report No 10

DSD lab

Submitted By: Syeda Afia Shah

Registration No: 22pwcse2115

Section: B

"On my honor , as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work"

Department of Computer Systems Engineering

University of Engineering and Technology Peshawar

Change the functionality of the lock such that it unlocks on the sequence of 11011.

```
//clock_divider
module clock_divider(
    input clk_in,
    output reg clk_out
);
   reg [27:0] counter = 28'd0;
            parameter divisor = 28'd1000000;

    always @(posedge clk_in) begin
                        counter <= counter + 28'd1;

                        if (counter >= (divisor - 1))
            counter <= 28'd0;

                        clk_out <=(counter < divisor/2)?1'b1:1'b0;
                        end
endmodule



//lock_fsm2
module lock_fsm2(
            input btn0,
            input btn1,
            input clk,
```

```
	input RST_BTN,
	output reg led,
	output reg [3:0]bcd
);
	reg [2:0]state;

	parameter s0 = 3'b000;
	parameter s1 = 3'b001;
	parameter s2 = 3'b010;
	parameter s3 = 3'b011;
	parameter s4 = 3'b100;
	parameter s5 = 3'b101;

	always@(posedge clk) begin
		if(RST_BTN)
			state <= s0;

		else
			case(state)
				s0: begin
					if(btn0)
						state <= s0;
					else if(btn1)
						state <= s1;
					else
						state <= s0;
```

```
end

s1: begin
        if(btn0)
                state <= s1;
        else if(btn1)
                state <= s2;
        else
                state <= s1;
end

s2: begin
        if(btn0)
                state <= s3;
        else if(btn1)
                state <= s2;
        else
                state <= s2;
end

s3: begin
        if(btn0)
                state <= s3;
        else if(btn1)
                state <= s4;
        else
```

```
                        state <= s3;
                end

                s4: begin
                        if(btn0)
                                state <= s4;
                        else if(btn1)
                                state <= s5;
                        else
                                state <= s4;
                end

                s5: begin
                        if(btn0)
                                state <= s5;
                        else if(btn1)
                                state <= s0;
                        else
                                state <= s5;
                end
        endcase
end



always@(*)
        case(state)
```

```
s0: begin
        led <= 0;
        bcd <= 4'b0000;
end

s1: begin
        led <= 0;
        bcd <= 4'b0001;
end


s2: begin
        led <= 0;
        bcd <= 4'b0010;
end

s3: begin
        led <= 0;
        bcd <= 4'b0011;
end

s4: begin
        led <= 0;
        bcd <= 4'b0100;
end
```

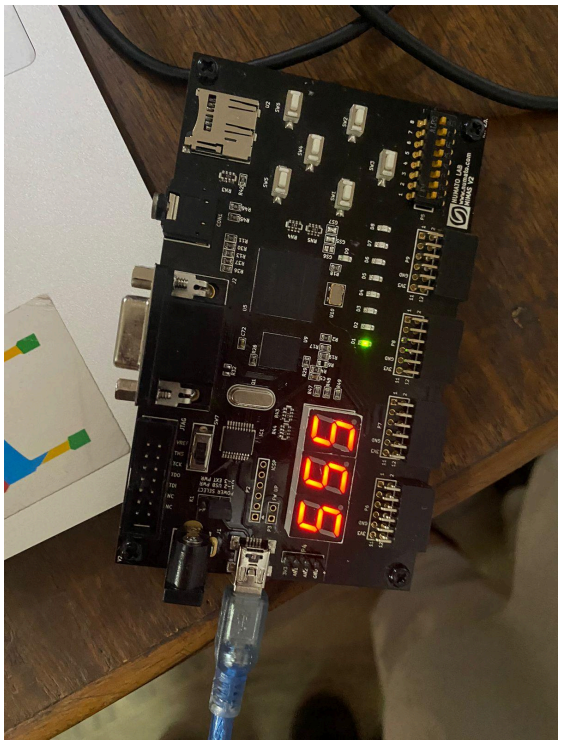
```
                s5: begin
                        led <= 1;
                        bcd <= 4'b0101;
                end
            endcase

endmodule



//synchronizer
module synchronizer(
        input clk,
        input btn,
        input rst,
        output synch_btn
    );

        wire Q1;

        D_FF df1(Q1, btn, clk, rst);
        D_FF df2(synch_btn, Q1, clk, rst);

endmodule

// D_FlipFLop
module D_FF(Q,D,clk,rst);
```

```
input D; // Data input
input clk; // clock input
input rst;
output reg Q; // output Q
always @(posedge clk or negedge clk)
begin
if(rst)
        Q = 1'b0;
else
        Q = D;
end
endmodule



//level_to_pulse
module level_to_pulse(
        input synch_input,
        input clk,
        input rst,
        output pulse
   );
         wire Q;

         D_FF df(Q, synch_input, clk, rst);
         and a(pulse ,~Q, synch_input);
endmodule
```

```
//btn_module

module btn_module(
        input btn,
        input CLK,
        input RST,
        output pulse
   );
        wire synch_btn;
        synchronizer s1(CLK, btn, RST, synch_btn);
        level_to_pulse lp1(synch_btn, CLK, RST, pulse);

endmodule



//BCD_to_SevenSeg

module BCD_to_SevenSeg(
   input [3:0] bcd,
        output reg [7:0] O
        );
```

```
	always @(*)
		case(bcd)	//gfedcba
			4'b0000: O = 8'b11000000;
			4'b0001: O = 8'b11111001;
			4'b0010: O = 8'b10100100;
			4'b0011: O = 8'b10110000;
			4'b0100: O = 8'b10011001;
			4'b0101: O = 8'b10010010;
			4'b0110: O = 8'b10000010;
			4'b0111: O = 8'b11111000;
			4'b1000: O = 8'b10000000;
			4'b1001: O = 8'b10010000;
			default: O = 8'b11111111;
		endcase

endmodule




//Top Module
module Top( CLK, RST, RST_BTN, BTN0, BTN1, LED, SEVENSEG);

	input CLK, RST, BTN0, BTN1, RST_BTN;
	output LED;
```

```
        output [7:0]SEVENSEG;
        wire SLOW_CLOCK;
        //wire synch_btn0, synch_btn1, synch_rst;
        wire pulse0, pulse1, RST_Pulse;
        wire [3:0]bcd;

        clock_divider divider(CLK, SLOW_CLOCK);
        lock_fsm2 my_fsm( pulse0, pulse1, SLOW_CLOCK, RST_Pulse, LED, bcd);
        btn_module b0(BTN0, SLOW_CLOCK, RST,  pulse0);
        btn_module b1(BTN1, SLOW_CLOCK, RST,  pulse1);
        btn_module b2(RST_BTN, SLOW_CLOCK, RST,  RST_Pulse);

    /*
        synchronizer s1(SLOW_CLOCK, BTN0, RST, synch_btn0);
        level_to_pulse lp1(synch_btn0, SLOW_CLOCK, RST, pulse0);
        synchronizer s2(SLOW_CLOCK, BTN1, RST, synch_btn1);
        level_to_pulse lp2(synch_btn1, SLOW_CLOCK, RST, pulse1);
        synchronizer s3(SLOW_CLOCK, RST_BTN, RST, synch_rst);
        level_to_pulse lp3(synch_rst, SLOW_CLOCK, RST, RST_pulse);
        */
        BCD_to_SevenSeg(bcd, SEVENSEG);

endmodule
```



2) Change the functionality of the lock such that it unlocks on the sequence of 01011

```
//clock_divider
module clock_divider(
    input clk_in,
    output reg clk_out
);
    reg [27:0] counter = 28'd0;
    parameter divisor = 28'd1000000;

    always @(posedge clk_in) begin
        counter <= counter + 28'd1;
```

```
        if (counter >= (divisor - 1))
            counter <= 28'd0;

        clk_out <=(counter < divisor/2)?1'b1:1'b0;
        end
endmodule
//lock_fsm2
module lock_fsm2(
    input btn0,
    input btn1,
    input clk,
    input RST_BTN,
    output reg led,
    output reg [3:0]bcd
    );
    reg [4:0]state;  // Changed to 5 states to recognize 01011

    parameter s0 = 5'b00000;
    parameter s1 = 5'b00001;
    parameter s2 = 5'b00010;
    parameter s3 = 5'b00011;
    parameter s4 = 5'b00100;
    parameter s5 = 5'b00101; // Added state s5
    // parameter s6 = 5'b00110; // Not needed for 01011, removed.

    always@(posedge clk) begin
```

```
if(RST_BTN)
    state <= s0;

else
    case(state)
        s0: begin
            if(btn0)
                state <= s1; // 0
            else if(btn1)
                state <= s0;
            else
                state <= s0;
        end

        s1: begin
            if(btn0)
                state <= s1;
            else if(btn1)
                state <= s2; // 1
            else
                state <= s0;
        end

        s2: begin
            if(btn0)
                state <= s3; // 0
```

```
    else if(btn1)
        state <= s0;
    else
        state <= s0;
end

s3: begin
    if(btn0)
        state <= s1;
    else if(btn1)
        state <= s4; // 1
    else
        state <= s0;
end

s4: begin
    if(btn0)
        state <= s5; // 1
    else if (btn1)
        state <= s0;
    else
        state <= s0;
end

s5: begin  // Added state for the final 1
    if(btn0)
```

```
                state <= s1;
            else if(btn1)
                state <= s0;
            else
                state <= s5;
        end
    endcase
end



always@(*)
  case(state)
    s0: begin
        led <= 0;
        bcd <= 4'b0000;
    end

    s1: begin
        led <= 0;
        bcd <= 4'b0001;
    end



    s2: begin
        led <= 0;
        bcd <= 4'b0010;
```

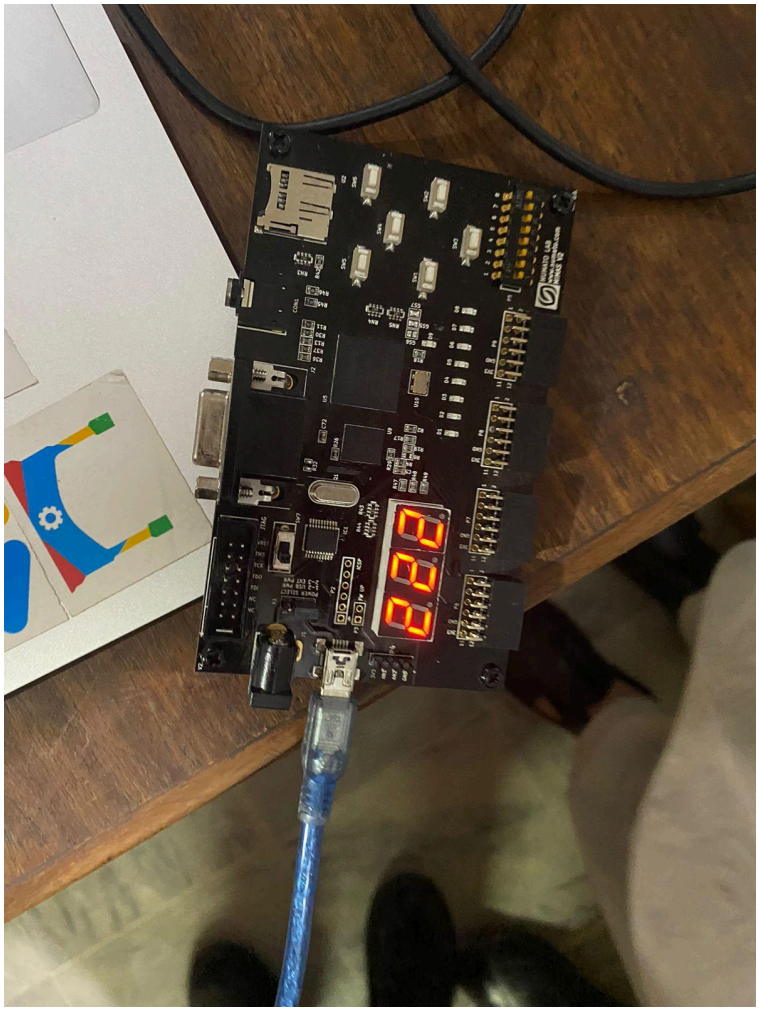
```
            end

            s3: begin
                led <= 0;
                bcd <= 4'b0011;
            end

            s4: begin
                led <= 0;
                bcd <= 4'b0100;
            end

            s5: begin
                led <= 1;  // LED is turned on when the sequence 01011 is detected.
                bcd <= 4'b0101;
            end
        endcase

endmodule



//synchronizer
module synchronizer(
    input clk,
    input btn,
    input rst,
```

```
    output synch_btn
    );

    wire Q1;

    D_FF df1(Q1, btn, clk, rst);
    D_FF df2(synch_btn, Q1, clk, rst);

endmodule

// D_FlipFLop
module D_FF(Q,D,clk,rst);
input D; // Data input
input clk; // clock input
input rst;
output reg Q; // output Q
always @(posedge clk or negedge clk)
begin
if(rst)
    Q = 1'b0;
else
    Q = D;
end
endmodule
```

```
//level_to_pulse
module level_to_pulse(
    input synch_input,
    input clk,
    input rst,
    output pulse
    );
     wire Q;

     D_FF df(Q, synch_input, clk, rst);
     and a(pulse ,~Q, synch_input);
endmodule
```

```
//btn_module

module btn_module(
    input btn,
    input CLK,
    input RST,
    output pulse
    );
    wire synch_btn;
```

```
    synchronizer s1(CLK, btn, RST, synch_btn);
    level_to_pulse lp1(synch_btn, CLK, RST, pulse);

endmodule



//BCD_to_SevenSeg

module BCD_to_SevenSeg(
    input [3:0] bcd,
    output reg [7:0] O
    );
    always @(*)
        case(bcd)        //gfedcba
            4'b0000: O = 8'b11000000;
            4'b0001: O = 8'b11111001;
            4'b0010: O = 8'b10100100;
            4'b0011: O = 8'b10110000;
            4'b0100: O = 8'b10011001;
            4'b0101: O = 8'b10010010;
            4'b0110: O = 8'b10000010;
            4'b0111: O = 8'b11111000;
            4'b1000: O = 8'b10000000;
            4'b1001: O = 8'b10010000;
            default: O = 8'b11111111;
        endcase
```

```
endmodule



//Top Module
module Top( CLK, RST, RST_BTN, BTN0, BTN1, LED, SEVENSEG);

    input CLK, RST, BTN0, BTN1, RST_BTN;
    output LED;
    output [7:0]SEVENSEG;
    wire SLOW_CLOCK;
    //wire synch_btn0, synch_btn1, synch_rst;
    wire pulse0, pulse1, RST_Pulse;
    wire [3:0]bcd;

    clock_divider divider(CLK, SLOW_CLOCK);
    lock_fsm2 my_fsm( pulse0, pulse1, SLOW_CLOCK, RST_Pulse, LED, bcd);
    btn_module b0(BTN0, SLOW_CLOCK, RST,  pulse0);
    btn_module b1(BTN1, SLOW_CLOCK, RST,  pulse1);
    btn_module b2(RST_BTN, SLOW_CLOCK, RST,  RST_Pulse);

    /*
    synchronizer s1(SLOW_CLOCK, BTN0, RST, synch_btn0);
```

```
    level_to_pulse lp1(synch_btn0, SLOW_CLOCK, RST, pulse0);
    synchronizer s2(SLOW_CLOCK, BTN1, RST, synch_btn1);
    level_to_pulse lp2(synch_btn1, SLOW_CLOCK, RST, pulse1);
    synchronizer s3(SLOW_CLOCK, RST_BTN, RST, synch_rst);
    level_to_pulse lp3(synch_rst, SLOW_CLOCK, RST, RST_pulse);
    */
    BCD_to_SevenSeg(bcd, SEVENSEG);

endmodule
```